

PAW

1er Cuatrimestre 2026

INSTRUCTOR: Sotuyo, Nico Manteca 🍷, Mateo, etc.

24 June 2026 - Ultima clase

Ejercicios clásicos y problemas comunes que vamos a enfrentar.

REST & HATEOAS. La gracia del final de paw es irse lo más al extremo posible de REST. La pet store original - que tienen en común estas URLs? (Ejemplo de la clase) Tienen verbos - son anti REST. **Las URLs identifican RECURSOS, NO ACCIONES.**

Sin convención, cada endpoint inventa su patrón. Por ejemplo, CamelCase mezclado con snake_case. Colecciones en plural + jerarquía clara.

Toda acción cabe en un POST... o no? POST /api?action=deletePet&id=10 ; POST /api?action=updatePet&id=10 ;

Cada método HTTP tiene una **semántica precisa**. Hay que ver qué cosas son idempotentes y que usamos en base a eso. No usar todo como GET y POST.

Problema muy común en finales: Lo que responde la API.

Siempre "200 OK en todo y en status lo que está mal" está mal. Si es un error hay que señalizarlo de alguna manera y si no lo es de otra manera. Ojo con la cache.

Códigos de estado semánticos

Es bueno retornar la locación donde está viviendo el recurso. Si haces 201 CREATED esta bueno que tenga esa información. El código HTTP es suficiente para saber si algo salió bien. El body solo existe cuando hay contenido que aportar.

HATEOAS

Hypermedia as the Engine of Application State: El servidor le dice al cliente que puede hacer a continuación. El cliente nunca hardcodea URLs. Sin HATEOAS, el cliente conoce de antemano cada URL y cada operación disponible. Si la API cambia, el cliente se rompe.

Con HATEOAS, el cliente sigue los links que el servidor incluye en cada respuesta. La API es autodescriptiva y navegable. HATEOAS se puede decir que es lo mas REST posible, la API te explica sola la API.

“El cliente conoce la API - La construyo así”

Atributo LINKS en la respuesta, donde se referencian los otros métodos. Caso particular: Método OPTIONS, donde la API te dice qué métodos están disponibles para ese recursos. “Por definir todos los requests mappings/controllers, jersey te lo responde en XML solo”.

En un mundo ideal (REST-HATEOAS) solo conocemos la URL base. Con HATEOAS no hace falta cambiar el front end cuando modificamos la API

Paginación: Problema común en los finales

Paginación en el body: ERROR.

El approach más intuitivo. (caso más comun). Toda la info esta ahi, por que seria un problema? Está mal el approach más común, porque no cumple con REST. **Paginación en el header Link, no en el body.**

Header link (RFC 8288)

El cliente itera siguiendo links, no hardcodea URLs. Cuando rel=“next” no hay más páginas.

Autenticación: Error común en los finales.

Esta mal tener un endpoint POST /login, POST /user/login.

No existe /login, la auth viaja en el header. HTTP Basic - Estandar del protocolo. Las credenciales viajan en el header *Authorization*, definido por el RFC 7617. Cualquier endpoint las acepta, no hay una URL especial de login.

Concepto de filter middleware. Te deja interceptar todos los requests. El middleware es para cosas transversales, NO para paginación, esta se encarga el endpoint.

Refresh Token: Logica de front end.

Flujo de access y refresh token. Refresh token lo estamos mandando todo el tiempo.

Diferencia entre 401 UNAUTHORIZED y 403 FORBIDDEN.

En uno es que no estás logueado/tus credenciales están vencidas y en el otro no tienes el permiso para acceder. No ponerle a todo uno o el otro porque es un error.

Nota: Un v1/v2 en un método es un error conceptual grave, ya que rompe el objetivo de REST. NUNCA usarlos. El header content-type tenes que poner un content type tuyo si quieres hacerlo, es decir, un JSON modificado.

El GET no tiene body. Un GET a /users deberia devolver una lista de usuarios. Igualmente, siguiendo HATEOAS no deberías saber que podes hacer eso.

Frontend + Spring

Por que cada paso existe - no solo como hacerlo.

En desarrollo: dos servidores. 2 mundos.

Frontend: npm start/ vite dev

Backend: mcu tomcat7:run

Hay problemas cuando se quiere hacer el .war. En producción hay un solo servidor para todo. Para deployar la app, hay que recordar correr 2 builds en orden correcto:

cd frontend → npm install → npm run build → mvn clean package

El frontEnd como Modulo Maven.

Proceso manual: fragil.

Frontend como modulo: un solo comando. Se puede hacer con un plugin de maven.

Con el plugin - 3 executions.

Problema: el módulo existe, maven no lo conoce. → Declarar en el POM padre.

Nuestro front end actualmente sigue siendo un módulo de maven. Hay que asegurarse que el front end se compile antes que webapp. Si esto pasa, no hay frontend en el war.

“Mvn primero resuelve las dependencias y luego el módulo”

El /dist existe, el WAR no lo sabe.

“Tenemos primero el módulo de frontend. En el frontend no existe los compilados de .class porque no es un proyecto de java. Todos los archivos viven en una carpeta. Primero los tienes que generar y después los tienes que incluir”.

URL del frontend != URL de la api

No hacer REST el frontend.

Se puede poner como error page 404 el index. No esta tan lindo igualmente.

La app se deploya en Tomcat bajo un contexto: localhost:8080/my-app/

Los bundlers tienen cada uno su archivo de configuración. A todas las URLs les prepeadea esa URL.

“Lo mejor que se le puede pasar a la IA es un RFC. Lo REST no es un RFC, es un paper. HATEOAS es un RFC. HTTP tiene su RFC”.

Devolución de la API.

No tendría como entidad fuerte token. Reemplazarlo por, en primer lugar por un un POST a users, que genera sers y genera via el mail el token. Via mail usar mimetypes. En el body mandar el user que estas queriendo logear/generar el token.

El registrar usuario esta bien

Esta mal el registration confirmation. Los endpoints deben ser entidades fuertes. /users, /comments. No verbos o resultados, como es el caso/. Post a users con un mimetype deberia ser su reemplazo.

Validar token de confirmacion mal. El token que mandas por mail, es uno que podes usar el eh headers para autenticar, de corta duracion por tema de seguridad. Cuando queramos verificar va a ser un patch a users, y se va a poder hacer ese patch porque suamos el token como header. Despues de eso, se puede devolver un acces token y refresh token, dependiendo de cuanto dura la sesion.

Consumir token -> cuenta ACTIVEpodria ser algo mas como /user/id/

Solicitar reset de password tambien tiene le mismo problema, tiene que ser un post a .usees con un mimetype, en el body se manda mail del usuario que quiere resetear y se manda un mail con el token y ese token se manda como auth.

LOS TOKENS NO SE CONSUMEN, LES DAS CORTA DURACION Y LISTO. SE MANDA POR HEADERS.

Volar todas las cuentas legacy. Se puede hacer un flujo manual con la que se crea su cuenta con user/password transitoria.

Activar cuenta reclamada: No hacer el flujo directamente

/Users bien

/users/me esta mal a menos que me sea tu ID

Si ya estas logueado, cambiar password es un PATCH. O sino por header. Como obtener el token para cambiar la password? Existe el post a users que lo manda.

Tambien locale, pero doble parametro en general no se usa

Get / esta bien. Business clock no lo ve mal. Para datos del sistema es mejor un endpoint /meta.

Cachear endpoint de images. Nunca usar un PUT para overridear un ID de una imagen y en gral el DELETE tmp se usa.

Para availability: No se hacen parametros sobre entidades fuertes en general. Queryparam usar.

Sobre una coleccion solo usar filtros y ordenamiento.

Reservation-slots, para estadísticas en capaz hacer un /stats. O usar un custom header que responda al GET /restaurants.

O bien, un get con un mimetype o restaurante.details+v1.json. Restaurant.v1+json.

Sacar el /time para que el backend del servidor, la deberian recibir del cliente.

RESERVATIONS, si es una entidad fuerte no puede aparecer despues de otra entidad fuerte. Si pasa eso, se rompe la unicidad.

Reservations si es un obj fuerte, el post debe ser a /RESERVATIONS, no a [/restaurantes.id/reservations](#).

Esta bien el [//api/reservations/{id}/confirmation](#)

La diferencia con auth es que auth no es una entidad. Cuando te quieres registrar, manejas logica de sesion, por eso se maneja por headers, y este se puede hacer a cualquier endpoint. En general se usa a /users o a /

CUIDADO CON LOS PUT SI ES QUE TIENEN EFECTOS SEVUNDARIOS. Por ej, mandar un mail, modificar otra tabla y no se vuelve idemp[otente, es mejor cambiarlo a un POST.

Adentro de reservation deberia haber una variable completion que pises su valor.

No es un PUT /lo que sea, sino seria un POST con un mimetype.

Chequear: Mesas a las que se puede reasignar la reserva. No es de los restaurantes?

/reservation-check-ins deberia ser camel case porque las URLs estan siendo camel case.

Definir si es una entidad fuerte o débil. Si es fuerte hay que cambiar como esta implementado.

Paginacion: `?page=` (1-based) + `?pageSize=`. Esta mal.

El /users/id/followers deberia devolver links. Que te pueda devolver a un redirect a /followers con queryparam de followed by ID.

Mandarle mensaje con esto ultimo de followers a Rana/Ranucci por discord.